중간고사 대체 과제

문제 1: 비트 연산과 형 변환을 활용한 권한 판별 프로그램 구현

비트 연산과 명시적 형 변환을 활용하여 사용자 권한을 설정하고 판별하는 프로그램을 구현한다.

세부 내용

- 권한은 다음과 같이 비트 값으로 정의한다.
  · 읽기 권한: 1, 쓰기 권한: 2, 실행 권한: 4, 삭제 권한: 8
- 사용자에게 부여할 권한 번호를 반복 입력받아 하나의 정수 변수에 비트 OR 연산으로 누적한다.
- 0을 입력하면 권한 설정을 종료한다.
- 설정 완료 후, 4개의 권한 각각에 대해 비트 AND 연산으로 포함 여부를 판별하여 출력한다.
- 판별 결과는 bool로 명시적 형 변환하여 true/false로 출력한다. (std::boolalpha 활용)
- 최종적으로 누적된 권한 값을 10진수와 2진수(bitset<8>)로 함께 출력한다.

출력

```
권한 설정 (1:읽기, 2:쓰기, 4:실행, 8:삭제, 0:설정 완료)
권한 입력: 1
권한 입력: 4
권한 입력: 0

[권한 판별 결과]
읽기 권한: true
쓰기 권한: false
실행 권한: true
삭제 권한: false
권한 값: 5 (00000101)
```

```
권한 설정 (1:읽기, 2:쓰기, 4:실행, 8:삭제, 0:설정 완료)
권한 입력: 2
권한 입력: 8
권한 입력: 2
권한 입력: 0

[권한 판별 결과]
읽기 권한: false
쓰기 권한: true
실행 권한: false
삭제 권한: true
권한 값: 10 (00001010)
```

문제 2: 반복문과 조건문을 활용한 구구단 출력 프로그램 구현
조건문과 반복문을 조합하여 다양한 조건으로 구구단을 출력하는 프로그램을 구현한다.

세부 내용
- 시작 단과 끝 단을 입력받는다.
- 시작 단이 끝 단보다 크면 "잘못된 입력입니다."를 출력하고 다시 입력받는다. (do-while 활용)
- 단의 범위는 2 이상 9 이하로 제한한다. 범위를 벗어나면 다시 입력받는다.
- 각 단을 출력할 때, 결과가 홀수인 곱셈만 출력한다. (continue 활용)
- 출력 중 결과 값이 50을 초과하면 해당 단의 출력을 중단하고 다음 단으로 넘어간다. (break 활용)
- 전체 출력이 끝나면 출력된 곱셈 결과의 총 개수를 출력한다.

출력

```
시작 단 입력: 7
끝 단 입력: 9

[7단] (홀수 결과만, 50 초과 시 중단)
7 x 1 = 7
7 x 3 = 21
7 x 5 = 35
7 x 7 = 49
-> 7단 중단 (50 초과)

[8단] (홀수 결과만, 50 초과 시 중단)
8 x 3 = 24
-> 8단 중단 (50 초과)

[9단] (홀수 결과만, 50 초과 시 중단)
9 x 1 = 9
9 x 3 = 27
9 x 5 = 45
-> 9단 중단 (50 초과)

출력된 곱셈 결과 총 개수: 7
```

문제 3: 예외 처리와 캡슐화를 활용한 은행 계좌 관리 프로그램 구현

try, catch, throw를 활용한 예외 처리와 클래스 캡슐화를 결합하여 은행 계좌를 관리하는 프로그램을 구현한다.

세부 내용

- Account 클래스를 정의한다.
  · 멤버 변수 (private): 계좌 소유자 이름(string), 잔액(int)
  · 멤버 함수 (public): 입금, 출금, 잔액 조회, 정보 출력
- 잔액은 생성 시 0으로 초기화한다.
- 입금 시 금액이 0 이하이면 throw로 정수형 예외를 던지고, catch(int)로 받아 "입금 오류: 0보다 큰 금액을 입력하세요."를 출력한다.
- 출금 시 금액이 잔액보다 크면 throw로 float형 예외를 던지고, catch(float)로 받아 "출금 오류: 잔액이 부족합니다. (현재 잔액: ___원)"을 출력한다.
- 그 외 예상치 못한 예외는 catch(...)로 처리하여 "알 수 없는 오류가 발생했습니다."를 출력한다.
- main 함수에서 Account 객체를 생성한 뒤, 메뉴(1:입금, 2:출금, 3:잔액 조회, 4:종료)를 반복 출력하여 사용자 입력에 따라 동작한다.
- 메뉴 선택은 switch 문으로 작성한다.
- 종료 선택 시 최종 계좌 정보를 출력하고 프로그램을 종료한다.

출력

```
계좌 소유자 이름 입력: 홍길동

===== 계좌 메뉴 =====
1. 입금
2. 출금
3. 잔액 조회
4. 종료
메뉴 선택: 1
입금 금액: 10000
입금 완료 (잔액: 10000원)

===== 계좌 메뉴 =====
1. 입금
2. 출금
3. 잔액 조회
4. 종료
메뉴 선택: 2
출금 금액: 3000
출금 완료 (잔액: 7000원)

===== 계좌 메뉴 =====
1. 입금
2. 출금
3. 잔액 조회
4. 종료
```

```
메뉴 선택: 2
출금 금액: 50000
출금 오류: 잔액이 부족합니다. (현재 잔액: 7000원)

===== 계좌 메뉴 =====
1. 입금
2. 출금
3. 잔액 조회
4. 종료
메뉴 선택: 1
입금 금액: -500
입금 오류: 0보다 큰 금액을 입력하세요.

===== 계좌 메뉴 =====
1. 입금
2. 출금
3. 잔액 조회
4. 종료
메뉴 선택: 3
[잔액 조회] 홍길동님의 잔액: 7000원

===== 계좌 메뉴 =====
1. 입금
2. 출금
3. 잔액 조회
4. 종료
메뉴 선택: 4

[최종 계좌 정보]
소유자: 홍길동
잔액: 7000원
프로그램을 종료합니다.
```

문제 4: 클래스를 활용한 학생 성적 관리 프로그램 구현

추상화와 캡슐화를 적용하여 학생 성적을 관리하는 클래스를 설계하고 구현한다.

세부 내용

- Student 클래스를 정의한다.
- 멤버 변수 (private): 이름(string), 과목 점수 3개(int 배열), 평균(double)
- 멤버 함수 (public): 이름 설정, 과목 점수 입력(3과목 반복 입력), 평균 계산, 정보 출력
- 멤버 변수는 반드시 private으로 선언하고, public 멤버 함수를 통해서만 접근한다.
- 과목 점수 입력 시 0~100 범위를 벗어나면 "유효하지 않은 점수입니다."를 출력하고 해당 과목을 다시 입력받는다.
- 평균 계산 시 명시적 형 변환을 사용하여 실수형으로 계산한다.
- 학생 객체를 3개 생성하여 각각 데이터를 입력받고, 전체 학생 정보를 출력한 뒤 평균이 가장 높은 학생의 이름을 출력한다.

출력

```
[학생 1 정보 입력]
이름: 홍길동
과목 1 점수: 85
과목 2 점수: 90
과목 3 점수: 78

[학생 2 정보 입력]
이름: 김철수
과목 1 점수: 92
과목 2 점수: 88
과목 3 점수: 95

[학생 3 정보 입력]
이름: 이영희
과목 1 점수: 105
유효하지 않은 점수입니다.
과목 1 점수: 76
과목 2 점수: 82
과목 3 점수: 90

======= 성적 결과 =======
이름: 홍길동 | 점수: 85, 90, 78 | 평균: 84.33
이름: 김철수 | 점수: 92, 88, 95 | 평균: 91.67
이름: 이영희 | 점수: 76, 82, 90 | 평균: 82.67

최고 평균 학생: 김철수 (91.67)
```

문제 5: 상속과 다형성을 활용한 탈것 관리 프로그램 구현

상속성과 다형성을 활용하여 다양한 탈것을 관리하는 프로그램을 구현한다.

세부 내용

- 부모 클래스 Vehicle을 정의한다.
  · 멤버 변수 (protected): 이름(string), 최대 속도(int), 현재 속도(int)
  · 멤버 함수 (public): 정보 출력, 가속(virtual), 정지
  · 가속 함수는 virtual로 선언하여 자식 클래스에서 override한다.
- 자식 클래스 3개를 정의한다.
  · Car: 가속 시 현재 속도에 20을 더한다. 최대 속도를 초과하면 최대 속도로 고정한다.
  · Bicycle: 가속 시 현재 속도에 5를 더한다. 최대 속도를 초과하면 최대 속도로 고정한다.
  · Airplane: 가속 시 현재 속도에 100을 더한다. 최대 속도를 초과하면 최대 속도로 고정한다.
- 각 자식 클래스의 가속 함수에서는 탈것 이름과 가속 방식을 출력한다.
  · 예: "[자동차] 엔진 가속: 현재 속도 40"
- main 함수에서 Vehicle 포인터 배열(크기 3)을 선언하고, 각각 Car, Bicycle, Airplane 객체를 동적 생성하여 대입한다.
- 반복문으로 포인터 배열을 순회하며 가속 함수를 3회씩 호출하고 정보를 출력한다.
- 부모 클래스 포인터로 호출했을 때 자식 클래스의 가속 함수가 실행됨을 확인한다.
- 프로그램 종료 전 동적 할당한 메모리를 모두 해제한다.

출력

```
===== 탈것 가속 테스트 =====

[자동차] 엔진 가속: 현재 속도 20
[자동차] 엔진 가속: 현재 속도 40
[자동차] 엔진 가속: 현재 속도 60
-> 자동차 | 최대 속도: 200 | 현재 속도: 60

[자전거] 페달 가속: 현재 속도 5
[자전거] 페달 가속: 현재 속도 10
[자전거] 페달 가속: 현재 속도 15
-> 자전거 | 최대 속도: 30 | 현재 속도: 15

[비행기] 엔진 풀파워 가속: 현재 속도 100
[비행기] 엔진 풀파워 가속: 현재 속도 200
[비행기] 엔진 풀파워 가속: 현재 속도 300
-> 비행기 | 최대 속도: 900 | 현재 속도: 300

===== 가속 추가 테스트 =====
[자전거] 페달 가속: 현재 속도 20
[자전거] 페달 가속: 현재 속도 25
[자전거] 페달 가속: 현재 속도 30
[자전거] 페달 가속: 최대 속도 도달 (30)
-> 자전거 | 최대 속도: 30 | 현재 속도: 30

메모리 해제 완료
프로그램을 종료합니다.
```

문제 6 종합 설계 문제

1~7주차에 학습한 내용을 종합하여 아래 요구사항을 만족하는 프로그램을 직접 설계하고 구현한다.

프로그램 주제

카페 주문 관리 프로그램을 구현한다. 메뉴 등록, 주문, 주문 내역 조회 기능을 포함하며, 프로그램은 사용자가 종료를 선택할 때까지 반복 실행된다.

필수 활용 요소

아래 항목을 프로그램 내에서 모두 사용해야 한다. 어디에 어떻게 적용할지는 자유롭게 결정한다.

- 클래스 1개 이상 (멤버 변수 private, 멤버 함수 public)
- 상속 관계 1개 이상 (부모-자식 클래스)
- 조건문(if 또는 switch)과 반복문(for, while, do-while 중 택1 이상)
- 명시적 형 변환 1회 이상
- try, catch, throw를 활용한 예외 처리 (아래 조건을 따른다)

예외 처리 조건

발생 상황	throw 형식	catch 처리 내용
주문 수량에 0 이하를 입력한 경우	throw 정수형 (입력된 수량)	catch(int)로 받아 "수량 오류: [값]은(는) 유효하지 않습니다." 출력
잔액이 부족하여 결제할 수 없는 경우	throw float형 (부족 금액)	catch(float)로 받아 "결제 오류: [금액]원이 부족합니다." 출력
그 외 예상치 못한 예외	-	catch(...)로 받아 "알 수 없는 오류가 발생했습니다." 출력

제출 요구사항

- 설계 설명서 (아래 내용을 포함하여 자유 형식으로 작성, 분량 제한 없음)
- 본인이 설계한 클래스 구조와 상속 관계를 간단히 설명
- 각 필수 활용 요소를 코드의 어느 부분에 적용했는지 명시
- 예외 처리가 동작하는 상황 설명